

```
# humane-explainerdashboard ,Â XAI API Reference
## RAG Knowledge Document for HumAIne Swarm Chatbot

**Library:** `humane-explainerdashboard` (HumAIne fork of
`explainerdashboard`)
**Current version:** 0.4.8
**Install:** `pip install explainerdashboard lime`
**Purpose:** Enables the HumAIne Swarm chatbot to answer questions about
ClassifierExplainer, SHAP, LIME, and the TextExplanationComponent ,Â
with code patterns that run directly in the hackathon notebook.
**HumAIne project:** Grant Agreement 101120218
```

1 ,Â Overview

`humane-explainerdashboard` wraps any scikit-learn compatible model with a full XAI pipeline:

1. **Compute** SHAP values, permutation importances, partial dependence data
2. **Render** these as interactive Plotly visualisations ,Â either inline in a notebook or as a full Dash web application
3. **Integrate** LIME text explanations via the `TextExplanationComponent` (text classification specific)

In the HumAIne context, this library satisfies the EU AI Act and D5.4 Module 3 explainability requirements: transparency, accountability, and the ability to audit individual AI routing decisions.

2 ,Â Core Classes

2.1 ClassifierExplainer

The primary entry point for classification tasks. Wraps a trained sklearn classifier and provides SHAP-based explanations.

```
`python
from explainerdashboard import ClassifierExplainer

explainer = ClassifierExplainer(
    model,                # trained sklearn classifier (e.g.
                           # LogisticRegression instance)
    X_test,                # pandas DataFrame with named feature columns ,Â
                           # NOT numpy array
    y_test,                # pandas Series of true integer labels
    labels=None,           # list of class name strings, e.g.
                           # ['Hardware', 'Software', 'Design', 'Electronics']
    cats=None,             # list of one-hot encoded feature groups to merge
                           # (optional)
    idxs=None,             # index labels for samples (optional, defaults to
                           # integer index)
```

```

        index_name=None, # display name for the index (optional)
        target=None,     # target variable name string (optional)
        descriptions=None, # dict of feature_name -> description (optional)
        precision='float64' # 'float32' saves memory for large datasets
    )
    ...

```

```

**Critical: why X_test must be a pandas DataFrame (not numpy array):**
- `ClassifierExplainer` uses column names for SHAP attribution plots
- Without column names, SHAP waterfall and importance plots show generic
  feature indices instead of meaningful names
- In the hackathon: `X_test_df = pd.DataFrame(X_test_np,
  columns=feature_names)` is prepared in Cell 2

```

```

**Critical: why pass `learner.estimator` not `learner` in the
hackathon:**
- `ClassifierExplainer` uses `isinstance()` checks to select the SHAP
  backend
- `modAL.ActiveLearner` is not a recognized sklearn estimator type
- Passing `learner.estimator` (the actual `LogisticRegression`) triggers
  `shap.LinearExplainer`, which is fast and exact
- Passing `learner` would trigger slow `shap.KernelExplainer` (minutes
  instead of seconds)

```

```

**SHAP backend auto-selection:**
| Model type | SHAP backend selected | Speed |
|---|---|---|
| LogisticRegression, LinearSVC, Ridge | `shap.LinearExplainer` | Fast |
| RandomForest, XGBoost, LightGBM | `shap.TreeExplainer` | Fast |
| Any other model | `shap.KernelExplainer` | Slow |

```

2.2 RegressionExplainer

Same interface as ClassifierExplainer for regression tasks. Uses
`shap.TreeExplainer` for tree models, `shap.LinearExplainer` for linear
regressors.

```

```python
from explainerdashboard import RegressionExplainer

explainer = RegressionExplainer(model, X_test, y_test, units="$",
target="Fare")
```

```

3 , which Key Plot Methods (ClassifierExplainer)

All plot methods return **Plotly Figure objects** , which display them with
`.show()`. No server or dashboard is needed for inline notebook use.

3.1 Global Feature Importance (SHAP)

```

```python

```

```
Mean absolute SHAP value per feature across all test samples
fig = explainer.plot_importances()
fig.show()

Detailed SHAP summary (beeswarm-style)
fig = explainer.plot_importances_detailed()
fig.show()

Permutation importance (model-agnostic alternative to SHAP)
fig = explainer.plot_importances(kind='permutation')
fig.show()
```

```

```
**What `plot_importances()` shows:**
- Bar chart of features sorted by mean |SHAP value|
- For TF-IDF text data: the words that most consistently influence team routing decisions
- Expected top features for HumAIne ticketing: `graphics`, `windows`, `hardware`, `circuit`, `bios`, `dos`
```

3.2 Individual Prediction Explanation (SHAP Waterfall)

```
```python
SHAP waterfall for sample at index 0
fig = explainer.plot_shap_contributions(idx=0)
fig.show()

For a specific class (pos_label=0 means class 0)
fig = explainer.plot_shap_contributions(idx=0, pos_label=0)
fig.show()
```
```

```
**What this shows:** How individual features (words) push the prediction up or down for the most likely class. Positive = evidence for this team. Negative = evidence against.
```

3.3 SHAP Dependence Plot

```
```python
How a single feature affects predictions across all samples
fig = explainer.plot_dependence('feature_name')
fig.show()
```
```

3.4 Confusion Matrix

```
```python
fig = explainer.plot_confusion_matrix()
fig.show()
```
```

3.5 ROC and PR Curves (binary/multi-class)

```
```python
```

```
fig = explainer.plot_roc_auc()
fig.show()
```

```
fig = explainer.plot_pr_auc()
fig.show()
```
```

4 ,Ã Displaying Plots in Colab vs JupyterLab

4.1 Colab Setup (recommended for hackathon)

```
```python
import plotly.io as pio
pio.renderers.default = 'colab'

Then simply:
fig = explainer.plot_importances()
fig.show()
or equivalently:
fig.show(renderer='colab')
```
```

4.2 JupyterLab / Kubeflow Notebooks

```
```python
import plotly.io as pio
pio.renderers.default = 'jupyterlab'

fig = explainer.plot_importances()
fig.show()
```
```

4.3 Save to HTML File (offline / sharing)

```
```python
import plotly.offline as pyo
pyo.plot(fig, filename='shap_importances.html', auto_open=False)
```
```

5 ,Ã ExplainerDashboard (Full Web App)

For a full interactive dashboard (not needed in the hackathon, but used in HumAIne deployments):

```
```python
from explainerdashboard import ExplainerDashboard

db = ExplainerDashboard(explainer)
db.run(port=8050)
```
```

```
# JupyterLab / Kubeflow inline mode
db.run(port=8050, mode='jupyterlab')
```

```
# External browser tab (Colab workaround)
db.run(port=8050, mode='external')
```

```
# Stop the server
db.terminate(port=8050)
```
```

```
Simplified single-page dashboard:
```python
db = ExplainerDashboard(explainer, simple=True)
db.run()
```
```

---

```
6 ,Äi TextExplanationComponent (Text Classification)
```

The `TextExplanationComponent` is the HumAIne-specific extension for text classification explanations. It combines the `ClassifierExplainer` with LIME to display highlighted text explanations in a dashboard.

```
```python
from explainerdashboard import ClassifierExplainer, ExplainerDashboard
from explainerdashboard.dashboard_components import
TextExplanationComponent
import pandas as pd
import numpy as np
```

```
# 1. Build the ClassifierExplainer (see Section 2.1)
explainer = ClassifierExplainer(model, X_test_df, y_test_series,
labels=class_names)
```

```
# 2. Define predict_fn ,Äi raw text ,Üi probability matrix
def predict_fn(texts):
    X_sparse = vectorizer.transform(pd.Series(texts).fillna(''))
    X_df = pd.DataFrame(X_sparse.toarray(), columns=feature_names)
    return model.predict_proba(X_df)
```

```
# 3. Prepare raw texts DataFrame (must match X_test_df index)
raw_texts_df = pd.DataFrame({'text': test_texts}, index=X_test_df.index)
```

```
# 4. Create TextExplanationComponent
text_component = TextExplanationComponent(
    explainer,
    predict_fn=predict_fn,
    raw_texts=raw_texts_df,
    text_col='text', # column name in
raw_texts_df
    class_names=class_names,
    num_features=15, # LIME top words to
highlight
```

```

        title='Ticket Routing Explanation',
        subtitle='Select a ticket and click Explain to view LIME word
contributions',
    )

# 5. Mount in dashboard and run
db = ExplainerDashboard(explainer, [text_component], title='HumAIne
Ticket XAI')
db.run(port=8050)
```

In notebook (inline mode):
```python
db.run(port=8050, mode='jupyterlab')    # Kubeflow
db.run(port=8050, mode='inline')        # Jupyter inline
```

7 ,Ã LIME Integration (Standalone, Without Dashboard)

For inline LIME in a notebook without a running server ,Ã this is the
approach used in the hackathon Cell 5.

```python
from lime.lime_text import LimeTextExplainer

# 1. Create explainer
lime_exp = LimeTextExplainer(class_names=TEAM_NAMES)

# 2. Explain one ticket
exp = lime_exp.explain_instance(
    text_string,          # raw text string (NOT vectorized)
    predict_fn,           # callable: list[str] -> np.ndarray shape (n,
n_classes)
    num_features=10,      # top N most influential words
    num_samples=1000,     # number of perturbations (default 5000, 1000 is
faster)
    top_labels=2          # explain for the top N predicted classes
)

# 3. Display in notebook (best option)
exp.show_in_notebook(text=True)    # highlights text + bar chart

# 4. Alternative: matplotlib bar chart
exp.as_pyplot_figure()
import matplotlib.pyplot as plt
plt.show()

# 5. Get raw weights as dict
weights = exp.as_list(label=predicted_class)
# Returns: [('word', weight), ...] sorted by |weight|
```

```

### ### 7.1 The predict\_fn Pattern for Text Models

This is the most important implementation detail for LIME on text data:

```
```python
def predict_fn(texts):
    """
    LIME-compatible inference for TF-IDF text classifier.

    LIME perturbs raw text by removing words. It calls this function
    with many perturbed strings. The function MUST reproduce the exact
    preprocessing pipeline used during training.

    Args:
        texts: list of str ,Ã raw ticket texts (may include None or '')

    Returns:
        np.ndarray of shape (len(texts), n_classes) ,Ã class
        probabilities
        Rows sum to 1.0
    """
    # Step 1: Vectorize with SAME fitted vectorizer (same vocabulary +
    IDF weights)
    X_sparse = vectorizer.transform(pd.Series(texts).fillna(''))
    # .fillna('') handles None/empty strings that LIME passes during
    perturbation

    # Step 2: Convert to dense DataFrame with named columns
    X_df = pd.DataFrame(X_sparse.toarray(), columns=feature_names)
    # .toarray() required: predict_proba needs dense array or DataFrame,
    not sparse

    # Step 3: Return probabilities (NOT class labels)
    return model.predict_proba(X_df)
    # Must return 2D array: shape (len(texts), n_classes)
    # Use predict_proba, never predict ,Ã LIME needs continuous
    probability values
```

Common mistakes that break LIME:
1. Forgetting `.fillna('')` ,Ã crash on None/empty strings
2. Forgetting `.toarray()` ,Ã TypeError (sparse matrix passed to
predict_proba)
3. Using `model.predict()` ,Ã returns integers, LIME needs floats
4. Using a freshly fitted vectorizer ,Ã different vocabulary than
training, wrong features
5. Returning wrong shape ,Ã must be (n_texts, n_classes), not (n_texts,)
or (n_classes,)

Verification:
```python
test = predict_fn(['test ticket text'])
assert test.shape == (1, n_classes)
assert abs(test.sum() - 1.0) < 0.01
```

```
...
```

```
---
```

8 ,Äî Persisting and Reloading Explainers

For production use ,Äî avoids recalculating SHAP values each time:

```
```python
Save
explainer.dump('explainer.joblib')

Reload
from explainerdashboard import ClassifierExplainer
explainer = ClassifierExplainer.from_file('explainer.joblib')
```
```

```
---
```

9 ,Äî Version Compatibility Notes (from Release History)

```
Version	Key change relevant to HumAIne
0.4.8	Latest ,Äî fixes deprecated `needs_proba` in `make_scorer`
0.4.6	Fixes SHAP 0.45 compatibility (3D array output for classifiers)
0.4.3	Pandas v2 now supported
0.4.0	Bootstrap 5 upgrade; Python 3.8+ required; Pipeline support
improved	
0.3.7	Manual SHAP value injection: `explainer.set_shap_values()`
0.3.6	Static HTML export: `dashboard.to_html()`,
`dashboard.save_html()`	
0.3.5	PyTorch support via skorch; `simple=True` simplified dashboard
0.3.0	Major API rename: `plot_shap_contributions()` ,Üí
`plot_contributions()` |
```

****Current recommended usage (v0.4.8):****

```
```python
Correct method names in v0.4.8:
explainer.plot_importances() # global feature importance
explainer.plot_shap_contributions() # individual waterfall (note: kept
this name in 0.4.x)
explainer.plot_dependence() # SHAP dependence
explainer.plot_interactions_importance() # interaction importance
```
```

```
---
```

10 ,Äî ExplainerHub (Multiple Dashboards)

For hosting multiple model dashboards together ,Äî used in HumAIne multi-pilot deployments:


```

```python
from explainerdashboard import ExplainerHub

hub = ExplainerHub([db1, db2, db3], title="HumAIne XAI Hub")
hub.run()

With access control
hub = ExplainerHub([db1, db2], logins=[('admin', 'password')])
hub.run()

Save and reload configuration
hub.to_yaml("hub.yaml")
hub2 = ExplainerHub.from_config("hub.yaml")
```

---

## 11 ,Ã WeakspotAnalyzer

Identifies regions of the input space where the model performs poorly ,Ã
useful for targeted AL improvement:

```python
from explainerdashboard import ClassifierExplainer

explainer = ClassifierExplainer(model, X_test_df, y_test_series,
labels=TEAM_NAMES)
WeakspotAnalyzer is accessed through the dashboard tab or directly:
ws = explainer.get_classification_df(cutoff=0.5)
```

---

*HumAIne EU Project ,Ã Grant Agreement 101120218*
*Library: humaine-explainerdashboard v0.4.8*
*For RAG indexing into HumAIne Swarm chatbot*

```